Module 7

Lab2: The Metronome Thing

B8019 – PROGRAMMING THE INTERNET OF THINGS M

Academic Year 2025/2026

Second cycle degree programme (LM) in Computer Engineering (cod. 5826)

Prof. Abdelsalam Helal



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA
DEPARTMENT
OF COMPUTER SCIENCE
AND ENGINEERING

© 2020-2026 Sumi Helal

What is a Metronome?

A device that generates a visual or audible pulse at consistent intervals. May generate both visual and audio simultaneously. Used as a pacemaker for playing music.

A common measurement for the speed, or tempo, of a metronome is Beats per Minute (BPM). A metronome at 60 BPM will pulse once every second, and at 120 BPM, twice every second, etc.



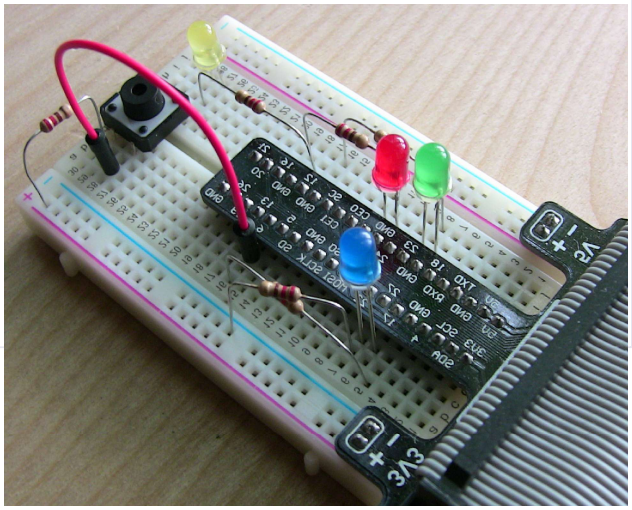


Setting the speed of the Metronome

In this assignment, you will set your Metronome by teaching it the desired tempo

> *You will repeatedly press a button at the desired tempo. The metronome will calculate the average time between the presses and set the BPM accordingly*

Requirements
(Bill of Materials)



Your Metronome will look like this. This is not a Metronome, however.

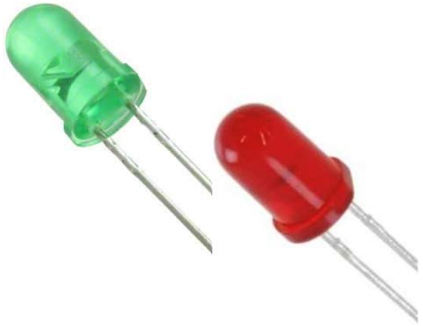
- Raspberry Pi and online manuals and programming tips.
- Your laptop
- One breadboard
- Jumper wires (male to male to use on the breadboard, and female to male jumper wires to connect your IO port on your Pi to the breadboard)
- Resistors
- **Two LED: one Green and one Red.**
- **Two pushbuttons**
- USB-x to Ethernet adaptor cable to connect your Pi to your laptop, or establish a wirless connection. Refer to "**Lab 1 Appendix on Configuring your Raspberry Pi Remote Connection.pdf"**.
- Optional: Additional material if you want to also generate Audio beats simultaneously with the LED.

Lab 2: Part I
Embedded Application

- You will create the Metronome as a Raspberry Hardware
- You will then create an externalized representation for the Metronome in the form of RESTful API
- The metronome should have two modes
  - **play** mode: which will blink an LED at a given tempo, and
  - **learn** mode: in which the user will repeatedly press a **push button (PB1)** to set the tempo.
  - The two modes will be toggled in between using another **push button (PB2)** dedicated to switching between play and learn.
- If you will do Audio, adjust the play mode accordingly.



Lab 2: Part I
Embedded Application



- In the **play** mode
  - The **green LED** will blink repeatedly at the previously set tempo. **On startup, the default mode should be at a tempo of 0, meaning the LED will always be off**.

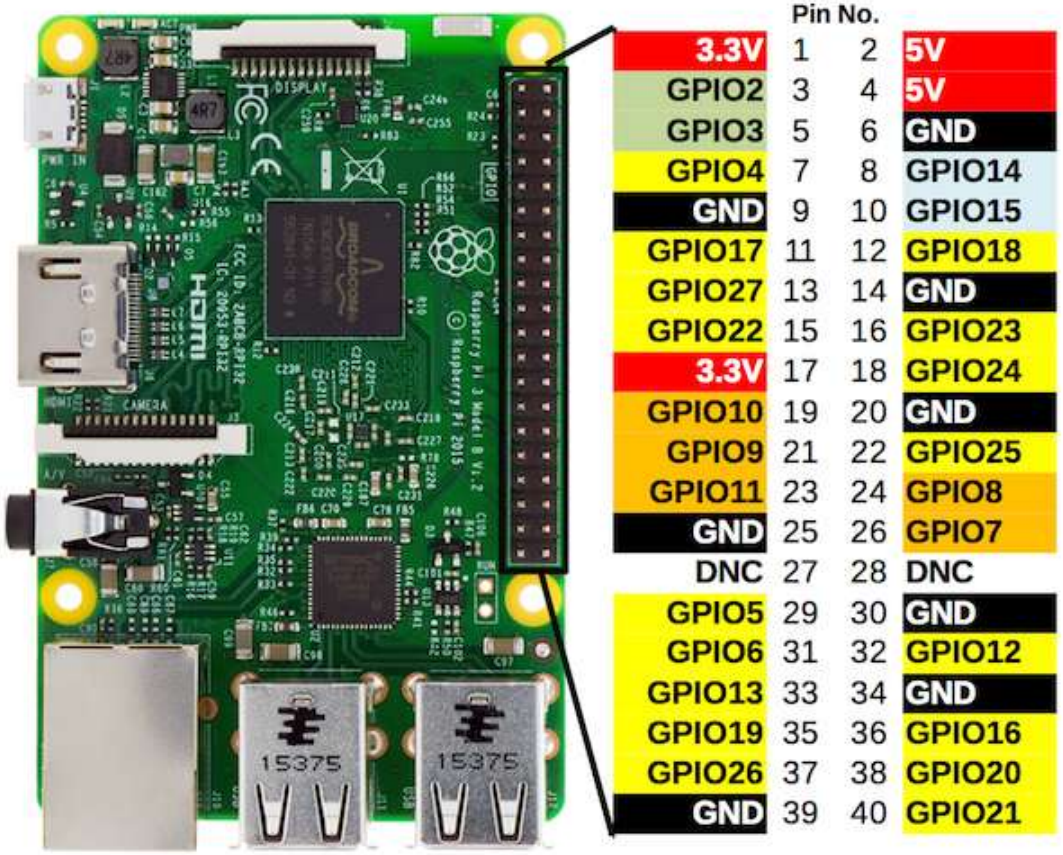
- In the **learn** mode
  - The **red LED** will pulse every time the user presses the button, and the time of the tap will be recorded. Once the user is done tapping (i.e., the user presses the mode button again), the BPM should be calculated and the new value should be reflected in the "play" mode (the green LED).
  - At least 4 taps are required to create a valid BPM measurement.

Lab 2: Part I
RESTful Services

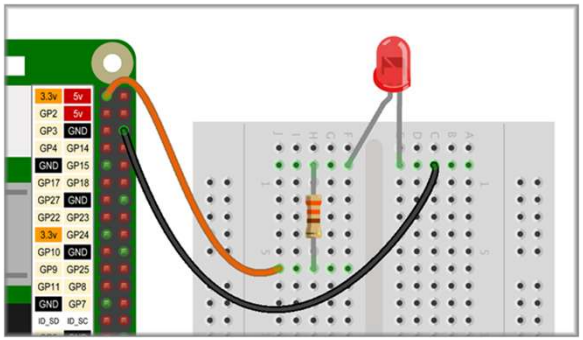
- You will create an **API for your Metronome Thing**
- You will do this by exposing the following REST **endpoints** in connection with **CRUD** Operations:
  - **/bpm/ (GET, PUT):** The current metronome BPM. Writing a new value here will update the speed the LED blinks at.
  - **/bpm/min/ (GET, DELETE):** Return the lowest BPM the user has set the metronome to. On delete, reset this value to 0.
  - **/bpm/max/ (GET, DELETE):** Works the same as the min endpoint, but tracks the highest BPM the user has set.
- Do not know REST: https://restapitutorial.com/ & YouTube Video: Learn RESR in 5 minutes

Connecting the Hardware

- Connecting the push buttons and the LED lights to your RPi: each pin is numbered so that it can be accessed through your software. See examples on how-to online: https://www.raspberrypi.com/documentation/computers/os.html
- ONLY USE THE 3.3V POWER PINS. The RPi GPIO senses at this voltage, so using 5V may damage the device.

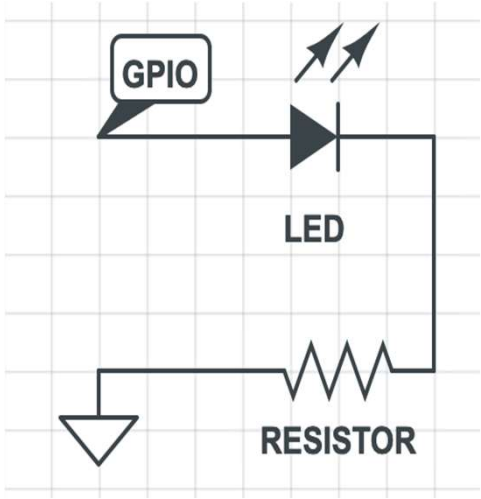


Connecting the LED

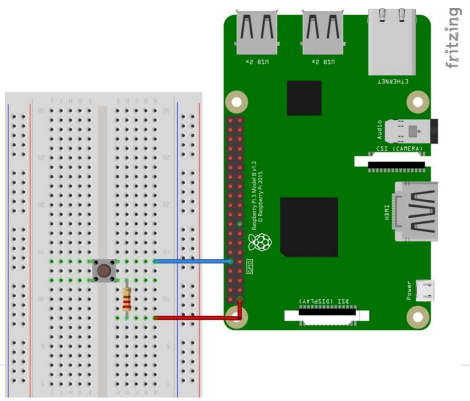


Your use of resistor to connect your Raspberry Pi to an LED light will look something like this. The above may not exactly what you will be doing.

- To connect the LED, a circuit must be created between an output GPIO pin and a GND pin. Note that if you plug the LED in directly, it will attempt to draw as much current as possible, and probably burn itself out.
- Therefore, an **appropriately-sized resistor** must be connected in series with the LED as shown in the exemplary circuit diagram to the right.
- Pay attention to the orientation of the anode (longer) / cathode (shorter) legs of the LED.

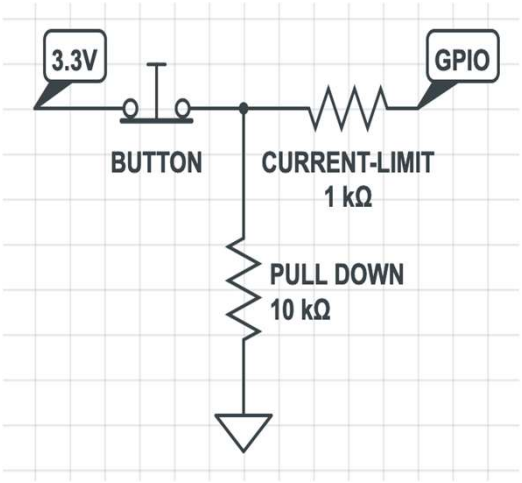


Connecting the Push Buttons



Your use of resistor to connect your Raspberry Pi to an LED light will look something like this. The above may not exactly what you will be doing.

- To connect the push button, a circuit must be created with 3.3V on one side, and an input GPIO pin on the other. This way, once the button is pushed, the 3.3V (“HIGH”) signal is connected to the input pin. In this configuration, however, the pin has no way to sense when it is not pressed, since it has no connection to GND (“LOW”)—this is called a “floating” input, and will return noise (random HIGH/LOW) in this state.
- To make sure the button always reads LOW when it is not pressed, we need **a pull-down resistor** connecting it to ground. The resistor makes sure the GPIO pin is always connected, but will be “overridden” by the "un-resisted” HIGH signal when pressed



Getting Started

- A Metronome shell is provided to get you started (unpack **metronome.zip** to your RPi)
- See & understand what it already implements for you
- Add a a working (undepreciated) Pi wiring library so you are able to reference the pins. Example: *sudo apt install wiringpi libcpprest-dev*

```
#include <chrono>
#include <thread>
using namespace std::chrono_literals;

#include <cpprest/http_msg.h>
#include <wiringPi.h>

#include "metronome.hpp"
#include "rest.hpp"

// ** Remember to update these numbers to your personal setup. **
#define LED_RED   17
#define LED_GREEN 27
#define BTN_MODE  23
#define BTN_TAP   24

// Mark as volatile to ensure it works multi-threaded.
volatile bool btn_mode_pressed = false;

// Run an additional loop separate from the main one.
void blink() {
    bool on = false;
    // ** This loop manages LED blinking. **
    while (true) {
        // The LED state will toggle once every second.
        std::this_thread::sleep_for(1s);

        // Perform the blink if we are pressed,
        // otherwise just set it to off.
        if (btn_mode_pressed)
            on = !on;
        else
            on = false;

        // HIGH/LOW probably equal 1/0, but be explicit anyways.
        digitalWrite(LED_RED, (on) ? HIGH : LOW);
    }
}

// This is called when a GET request is made to "/answer".
void get42(web::http::http_request msg) {
    msg.reply(200, web::json::value::number(42));
}

// This program shows an example of input/output with GPIO, along with
// a dummy REST service.
// ** You will have to replace this with your metronome logic, but the
//    structure of this program is very relevant to what you need. **
int main() {
    // WiringPi must be initialized at the very start.
    // This setup method uses the Broadcom pin numbers. These are the
    // larger numbers like 17, 24, etc, not the 0-16 virtual ones.
    wiringPiSetupGpio();

```

Implementing metronome.hpp

- Note that the base project also contains the header ***metronome.hpp***. This contains the definitions for the functionality of the metronome itself.
- You will need to write the implementation of this class. The metronome should **start_timing()** in the "learn" mode and ***stop_timing()*** upon return to the "play" mode.
- While timing, each press of the button should record the current time. The ***m_beats*** array can be used to implement a running average—only the most recent **N** taps are recorded.

Compiling the Metronome

Be sure to specify the LED and button pin numbers that correspond to your physical circuit setup. Once configured, you can compile the program with the following command (on your Raspberry Pi, inside the cloned folder):

```
mkdir build && cd build
cmake ..
make
```

The program can then be run using this command:

```
./main
```

Note that on subsequent builds, only make needs to be run. The sample also exposes a simple REST service that should be accessible through the same IP you use to SSH into your device, on port 8080.

Lab 2: Part II
HTML Dashboard for your Metronome

- For this part of the lab, you will create a web dashboard application to interact with your embedded service.
- The application will allow you to easily interact with your REST endpoints.
- Therefore, your web application should allow you to:
  - Refresh the current BPM, along with the minimum and maximum.
  - Set a new BPM on the device.
  - Reset the current minimum and maximum BPM.

Your Dashboard

- The dashboard should be implemented as a simple HTML and JavaScript project.
- It should present some simple styling to visualize the BPM values and allow the user to interact with the API through a set of buttons. While simple, use some “skins” to make the dashboard intuitive to use
- No specific format or layout is necessary, but the user should be able to interact with all endpoints offered by the RPi through the dashboard.



Submission ad Demonstration Requirements

- Submit a *.zip* file containing a separate folder for each part.
  - For part one, **delete** the build folder containing your compilation artifacts, and rename the top-level metronome folder to **part1**.
  - For part two, name the directory containing your HTML, JavaScript, and any supporting files **part2**.
  - The final *.zip* submission should contain these two folders along with a **pdf** detailing the relative contributions of your group members.
- In-class demos will be scheduled right after the due date.
  - **Keep your lab materials assembled and ready to deliver the demo.**